

REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION
P.O BOX 63.



RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST
B.P. 63.

Group 23

FACULTY OF ENGINEERING TECHNOLOGY

Report on the
Database Design and Implementation of a QoE Application
(NETLINQ)

Course Title: Internet Programming and Mobile App Development

Course code: CEF440

Course Instructor: Dr Nkemeni Valery

Date: June, 2026

Group 23 Members

NAME	MATRICULE
1. THELMA MARLEY NDAYINU	FE23A171
2. EKWEMBWE NKWELLE JUNIOR	FE23A044
3. MBUH ASSURANCE MULUH PILA	FE23A091
4. NYUH DELBERT KIMBI	FE23A129
5. YASIRI TEMWA GASTON	FE23A175

Table of Contents

Table of Contents	3
Overview	5
1. Data Elements	5
1.1 Core Business Entities	5
1.2 Network Metric — Field Definitions.....	5
1.3 QoE Feedback — Field Definitions.....	6
1.4 DataStore Preferences	6
1.5 Data Element Rules.....	7
2. Conceptual Design	8
2.1 Design Goals.....	8
2.2 Entity Relationships (Conceptual)	8
2.3 Two-Tier Storage Model.....	8
2.4 Data Lifecycle.....	8
2.5 Operator Analytics (Conceptual)	9
3. ER Diagram	10
3.1 Local Database (Room — On Device)	10
3.2 Cloud Database (Supabase — PostgreSQL).....	10
3.3 DataStore Entity	10
3.4 Analytics Views	11
4. Database Implementation.....	12
4.1 Local Database — Room (SQLite).....	12
Tables.....	12
Schema Migrations	12
DAO Operations	12
Key Files	12
4.2 Cloud Database — Supabase (PostgreSQL).....	13
Constraints	13
Indexes	13
Row Level Security (RLS).....	13
4.3 Device Preferences — DataStore.....	13
5. Backend Implementation	15
5.1 Architecture.....	15
5.2 Data Collection (monitoring/).....	15
Detection Thresholds	15

5.3 Repositories (data/repository/)	16
5.4 Identity and Hashing	16
5.5 Synchronisation (sync/)	16
Sync Triggers	16
5.6 Remote API (data/remote/)	16
5.7 Backend Folder Map	17
5.8 SRS Backend Requirements Status	17
6. Connecting Database to Backend	18
6.1 Connection Overview	18
6.2 Room Connection (Local)	18
Step 1 — Hilt provides database (DatabaseModule.kt)	18
Step 2 — DAO injected into repository	18
Step 3 — UI observes via Flow	18
Step 4 — Entity mapper converts Room ↔ Domain	19
6.3 Supabase Connection (Cloud)	19
Step 1 — Credentials from build config	19
Step 2 — Retrofit client (NetworkModule.kt)	19
Step 3 — SyncRepository uploads unsynced rows	19
Step 4 — WorkManager runs sync in background	19
6.4 DataStore Connection (Preferences)	19
6.5 End-to-End: Check Now → Sync → Supabase	20
6.6 End-to-End: Feedback Linked to Reading	20

Overview

NetLinq collects automatic network readings and user star ratings to measure mobile internet Quality of Experience (QoE) in Cameroon. The backend is offline-first: data is saved on the device first using Room, then uploaded to Supabase when connectivity is available. Users remain fully anonymous — no login or personal data is required; only a hashed device ID is transmitted to the cloud.

1. Data Elements

Data elements are the named fields the system stores, where they live, and what they mean.

1.1 Core Business Entities

Entity	Description	Stored Where
Network Metric	One automatic reading: signal, network type, latency, device info, timestamp	Room + Supabase
QoE Feedback	User star ratings (5 categories) plus optional link to a reading	Room + Supabase
Device Identity	Random UUID on device; SHA-256 hash uploaded as device_hash	DataStore (local) + Supabase (hash only)
App Preferences	Monitoring on/off, sync settings, trigger toggles	DataStore (local only, not synced)

1.2 Network Metric — Field Definitions

Field	Type	Required	Description
id	Long (local) / UUID (cloud)	Yes	Primary key
device_hash	Text	Cloud only	SHA-256 hash of local UUID — anonymous contributor ID
client_metric_id	Long	Cloud only	Local Room id — links feedback to this reading in Supabase
signal_strength	Integer (dBm)	No	Cellular signal from TelephonyManager (e.g. -85)
signal_quality	Integer	No	RSRP or quality indicator
network_type	Text	Yes	2G, 3G, 4G, 5G, WiFi, or None
latency_ms	Integer	No	Round-trip time in milliseconds (HEAD request)
device_model	Text	Yes	e.g. Pixel 7

Field	Type	Required	Description
android_version	Text	Yes	e.g. 14
recorded_at	Timestamp	Yes	When the reading was taken
synced	Boolean	Local only	false until successfully uploaded

1.3 QoE Feedback — Field Definitions

Field	Type	Required	Description
id	Long / UUID	Yes	Primary key
device_hash	Text	Cloud only	Same anonymous ID as metrics
overall_rating	Integer 1–5	Yes	Overall experience
responsiveness_rating	Integer 1–5	Yes	Internet responsiveness
streaming_rating	Integer 1–5	Yes	Streaming quality
call_quality_rating	Integer 1–5	Yes	Call quality
satisfaction_rating	Integer 1–5	Yes	General satisfaction
trigger_event	Text	No	What caused the prompt (e.g. signal drop)
network_type	Text	No	Network at rating time
network_metric_id	Long	No	FK to linked reading (local Room id)
signal_strength_snapshot	Integer	No	Signal copied at rating time
latency_ms_snapshot	Integer	No	Latency copied at rating time
notes	Text	No	Optional user comment
recorded_at	Timestamp	Yes	When feedback was submitted
synced	Boolean	Local only	false until successfully uploaded

1.4 DataStore Preferences

These values are stored locally only and are never synced to the cloud.

Key	Type	Purpose
device_id	UUID string	Local anonymous identity (never uploaded raw)
onboarding_complete	Boolean	User finished setup
monitoring_enabled	Boolean	Background collection on/off
wifi_only_sync	Boolean	Sync only on WiFi vs any network
feedback_frequency	Int	Prompt cooldown (5 / 15 / 30 minutes)
trigger	Boolean	Which events trigger feedback prompts

1.5 Data Element Rules

- Ratings must be 1–5 (enforced in Supabase with CHECK constraints).
- No name, email, phone number, or raw UUID in cloud tables.
- synced = false rows are the upload queue; after success they become synced = true.
- Synced local rows older than 7 days are deleted (FR12); cloud data remains.

2. Conceptual Design

Conceptual design describes what the system stores and how entities relate, before any SQL or Kotlin code.

2.1 Design Goals

Goal	Design Choice
Offline-first	Phone is source of truth while offline; cloud is backup + analytics
Privacy	Anonymous UUID → hash; no accounts
Link ratings to context	Feedback can reference the reading that triggered it
Minimal cloud exposure	Mobile app can INSERT only; operators use aggregated views later
Battery efficiency	Batch sync every 15 min; local purge after 7 days

2.2 Entity Relationships (Conceptual)

The system has two primary entities that share an anonymous device identity:

- One Network Metric can have zero or many QoE Feedback rows pointing to it.
- Feedback without a trigger link still stores snapshots (signal, latency) at rating time.
- Device hash groups all data from one installation without revealing identity.

2.3 Two-Tier Storage Model

Tier	Technology	Role
Tier 1 — Device	Room (SQLite) + DataStore	Fast local read/write; works offline
Tier 2 — Cloud	Supabase (PostgreSQL)	Persistent backup; operator analytics

The same logical entities exist in both tiers. The cloud adds device_hash, client_metric_id, and UUID primary keys.

2.4 Data Lifecycle

Step	Action
1	Collect reading or feedback from device sensors
2	Save to Room (synced = false)
3	User sees dashboard/history immediately
4	WorkManager or manual Sync triggered

Step	Action
5	POST to Supabase via REST API
6	Mark synced = true in Room
7	After 7 days: delete synced rows locally (FR12)

2.5 Operator Analytics (Conceptual)

Raw rows are not shown to operators on mobile. Supabase views aggregate by hour and network type, providing average QoE scores, average latency and signal, and feedback joined to linked readings. A future web dashboard will query these views with authenticated login.

3. ER Diagram

The NetLinq ER diagram spans four zones: DataStore, Room (local SQLite), Supabase (PostgreSQL cloud), and Analytics views. The full interactive diagram is provided in the accompanying HTML file ([netlinq-er-minimal.html](#)).

3.1 Local Database (Room — On Device)

Two tables with a 1-to-many optional relationship:

Entity	Fields	Notes
NETWORK_METRICS	id (PK), signalStrength, signalQuality, networkType, latencyMs, deviceModel, androidVersion, recordedAt, synced	Auto-collected readings
QOE_FEEDBACK	id (PK), overallRating, responsivenessRating, streamingRating, callQualityRating, satisfactionRating, triggerEvent, networkType, networkMetricId (FK), recordedAt, synced	User-submitted star ratings

Cardinality: One metric → zero or many feedback rows. FK rule: qoe_feedback.networkMetricId → network_metrics.id, ON DELETE SET NULL.

3.2 Cloud Database (Supabase — PostgreSQL)

The cloud schema mirrors the local schema with additions for anonymisation and deduplication:

Entity	Extra Cloud Fields	Notes
NETWORK_METRICS_CLOUD	id (UUID PK), device_hash, client_metric_id, created_at	Immutable after insert; UNIQUE(device_hash, client_metric_id)
QOE_FEEDBACK_CLOUD	id (UUID PK), device_hash, network_metric_id, metric_recorded_at, signal_strength_snapshot, latency_ms_snapshot, created_at	CHECK constraints on all rating fields (1–5)

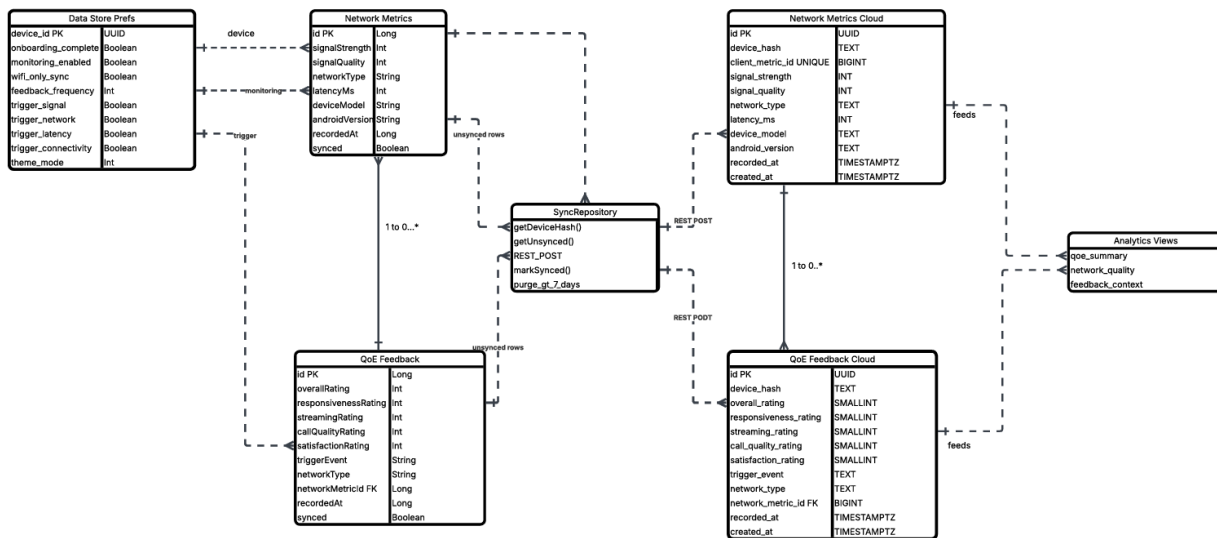
Cloud join rule: qoe_feedback.network_metric_id = network_metrics.client_metric_id AND same device_hash ensures joins are scoped to a single device.

3.3 DataStore Entity

The DataStore zone holds app preferences and the local anonymous UUID. It feeds the device_hash used as a grouping key in both Room and Supabase. It is not a SQL table and does not appear in Supabase.

3.4 Analytics Views

View	Purpose
operator_qoe_summary	Hourly average rating by network type
operator_network_quality	Hourly average latency/signal by network type
operator_feedback_with_context	Feedback joined to linked metric for operator drill-down



4. Database Implementation

4.1 Local Database — Room (SQLite)

Item	Detail
Database name	netlinq.db
Version	3
ORM	Android Room
Location	App private storage on device

Tables

- network_metrics — from NetworkMetricEntity.kt
- qoe_feedback — from QoeFeedbackEntity.kt (with FK to metrics)

Schema Migrations

Version	Change
1 → 2	Added linking columns on qoe_feedback
2 → 3	Added FK + index on networkMetricId

DAO Operations

Operation	SQL Behaviour
Insert	New row, synced = false
Observe all	Flow for UI (dashboard, history)
Get unsynced	WHERE synced = 0 for upload
Mark synced	UPDATE ... SET synced = 1
Purge	DELETE synced rows older than cutoff

Key Files

```
data/local/NetLinqDatabase.kt
data/local/entity/NetworkMetricEntity.kt
data/local/entity/QoeFeedbackEntity.kt
data/local/dao/NetworkMetricDao.kt
data/local/dao/QoeFeedbackDao.kt
di/DatabaseModule.kt
```

4.2 Cloud Database — Supabase (PostgreSQL)

The full schema is defined in docs/supabase/schema.sql and applied via the Supabase SQL Editor.

Constraints

- UNIQUE (device_hash, client_metric_id) on metrics — prevents duplicate uploads
- CHECK ratings BETWEEN 1 AND 5 on all five rating columns

Indexes

- recorded_at, network_type, device_hash on both tables
- Composite (device_hash, network_metric_id) on feedback

Row Level Security (RLS)

Policy	Role	Permission
Allow anonymous insert on network_metrics	anon	INSERT only
Allow anonymous insert on qoe_feedback	anon	INSERT only

Fetch Sample Operator QoE Summary Rows

User Profiles, Money Transactions

```
1 SELECT * FROM operator_qoe_summary LIMIT 10;
```

Results Explain Chart Export

Success. No rows returned

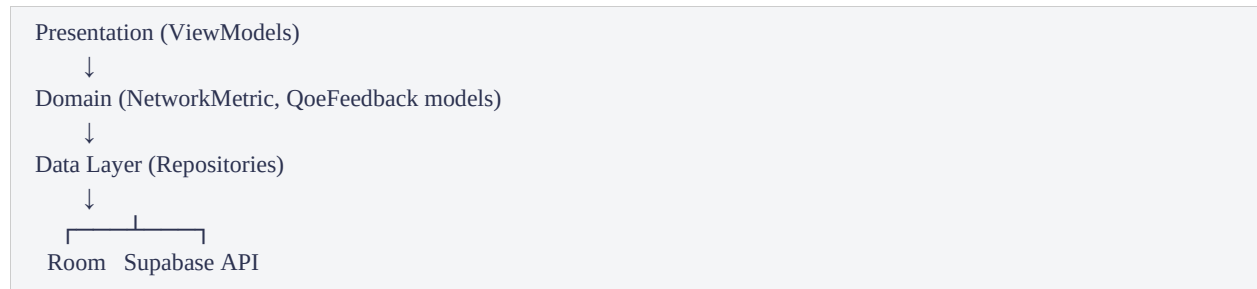
4.3 Device Preferences — DataStore

5. Backend Implementation

The backend encompasses the server-side and data-layer code that collects, stores, and syncs data — distinct from the Compose UI layer.

5.1 Architecture

NetLinq follows the Repository pattern with Hilt dependency injection across four layers:



5.2 Data Collection (monitoring/)

Component	File	Responsibility
NetworkMonitorService	NetworkMonitorService.kt	One reading: type + signal + latency → save
SignalStrengthCollector	SignalStrengthCollector.kt	dBm via TelephonyManager
NetworkTypeDetector	NetworkTypeDetector.kt	2G–5G / WiFi
LatencyMeasurer	LatencyMeasurer.kt	HEAD request latency (ms)
NetworkDegradationDetector	NetworkDegradationDetector.kt	Detect signal drop, type change, latency spike
NetworkMonitoringManager	NetworkMonitoringManager.kt	60s poll + network callbacks

Detection Thresholds

Event	Rule
Signal drop	Current dBm $\leq$ previous – 10
Network change	e.g. 4G → 3G
Latency spike	+200 ms and $\geq 2 \times$ previous
Connectivity loss	Online ↔ offline

5.3 Repositories (data/repository)

Repository	Methods	Backing Store
DeviceRepository	getOrCreateDeviceId(), getDeviceHash()	DataStore
NetworkMetricRepository	save, observe, getUnsynced, markSynced, purge	Room
QoeFeedbackRepository	save, observe, getUnsynced, markSynced, purge	Room
SyncRepository	syncAll(), purgeOldRecords(), isConfigured()	Room + Supabase

5.4 Identity and Hashing

The device UUID is generated locally and stored in DataStore. It is never transmitted in raw form. For each upload, the UUID is hashed with SHA-256 to produce device_hash — a one-way pseudonymous identifier.

UUID in DataStore → SHA-256 → device_hash on every upload

5.5 Synchronisation (sync/)

Component	File	Role
SyncRepository	data/repository/SyncRepository.kt	Build payloads, POST, mark synced, purge
SyncWorker	sync/SyncWorker.kt	WorkManager background job
SyncScheduler	sync/SyncScheduler.kt	Periodic work every 15 minutes
AppInitializer	AppInitializer.kt	Reschedule sync on app cold start

Sync Triggers

- WorkManager every 15 minutes (WiFi-only by default)
- Manual Sync button on dashboard
- Onboarding complete event
- Settings WiFi-only toggle change

5.6 Remote API (data/remote/)

Endpoint	Method	Payload
/rest/v1/network_metrics	POST	List of metrics + device_hash + client_metric_id
/rest/v1/qoe_feedback	POST	List of feedback + linking fields

HTTP client: Retrofit + OkHttp (di/NetworkModule.kt). Authentication: Supabase anon key in headers (apikey and Authorization).

5.7 Backend Folder Map



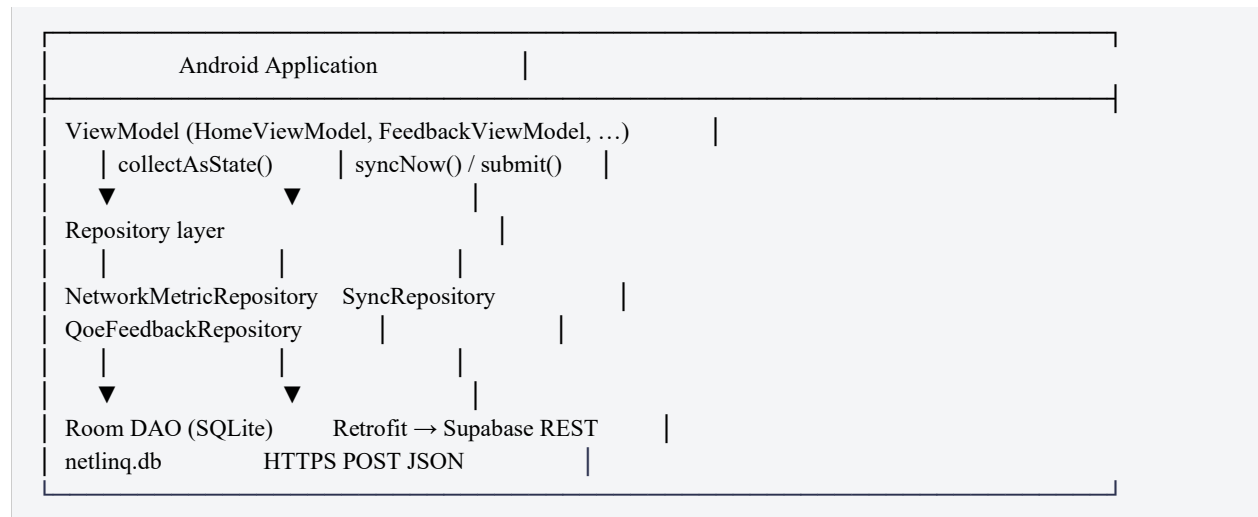
5.8 SRS Backend Requirements Status

ID	Requirement	Status
FR1	Anonymous device ID	Done
FR7	Local Room storage	Done (v3)
FR8	Offline operation	Done
FR9	Supabase sync	Done
FR11	Operator reports	Partial — SQL views done; web UI future
FR12	7-day local purge	Done

6. Connecting Database to Backend

This section describes how application code talks to each database tier.

6.1 Connection Overview



6.2 Room Connection (Local)

Step 1 — Hilt provides database (`DatabaseModule.kt`)

```
Room.databaseBuilder(context, NetLinqDatabase::class.java, "netlinq.db")
    .addMigrations(MIGRATION_1_2, MIGRATION_2_3)
    .build()
```

Step 2 — DAO injected into repository

```
@Singleton
class NetworkMetricRepository @Inject constructor(
    private val dao: NetworkMetricDao
) {
    suspend fun save(metric: NetworkMetric): Long =
        dao.insert(metric.toEntity())
}
```

Step 3 — UI observes via Flow

```
// HomeViewModel.kt
networkMetricRepository.observeMetrics().collect { metrics ->
    _uiState.update { it.copy/latestMetric = metrics.firstOrNull(), ... }
}
```

Step 4 — Entity mapper converts Room ↔ Domain

EntityMappers.kt provides `NetworkMetricEntity.toDomain()` and `NetworkMetric.toEntity()` for clean separation between persistence and domain layers.

6.3 Supabase Connection (Cloud)

Step 1 — Credentials from build config

```
buildConfigField("String", "SUPABASE_URL", ...) // from local.properties
buildConfigField("String", "SUPABASE_ANON_KEY", ...)
```

Step 2 — Retrofit client (NetworkModule.kt)

```
Retrofit.Builder()
    .baseUrl(SUPABASE_URL + "/")
    .client(okHttpClient) // adds apikey + Authorization headers
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(SupabaseApi::class.java)
```

Step 3 — SyncRepository uploads unsynced rows

```
val deviceHash = deviceRepository.getDeviceHash()
val metrics = networkMetricRepository.getUnsynced()
val response = supabaseApi.uploadNetworkMetrics(payload)
if (response.isSuccessful) {
    networkMetricRepository.markSynced(metrics.map { it.id })
}
```

Step 4 — WorkManager runs sync in background

```
// SyncWorker.kt
syncRepository.purgeOldRecords() // 7-day cleanup
syncRepository.syncAll()
```

6.4 DataStore Connection (Preferences)

```
// AppPreferences.kt
private val Context.dataStore by preferencesDataStore(name = "netlinq_prefs")

suspend fun ensureDeviceId(): String // creates UUID if missing
```

Used by `DeviceRepository` (hash for upload) and `AppInitializer` (sync bootstrap).

6.5 End-to-End: Check Now → Sync → Supabase

Step	Layer	What Happens
1	UI	User taps Check now on dashboard
2	ViewModel	HomeViewModel.collectNow()
3	Monitoring	NetworkMonitorService.collectAndSave()
4	Repository	NetworkMetricRepository.save() → Room INSERT
5	UI	Dashboard updates via Flow (synced = false)
6	User	Taps Sync
7	SyncRepository	Reads unsynced rows, maps to JSON, POST to Supabase
8	Room	markSynced(ids)
9	Supabase	Row visible in Table Editor with device_hash

6.6 End-to-End: Feedback Linked to Reading

Step	What Happens
1	Network event triggers prompt with metricId from Room
2	User submits quick rating
3	FeedbackViewModel saves QoeFeedback with networkMetricId + snapshots
4	Room FK validates link to network_metrics.id
5	Sync uploads feedback with network_metric_id = local Room id
6	Supabase join: network_metric_id = client_metric_id AND same device_hash